

Defining and calling tools in the API

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 10 of 12 · English notes

The live notebook walked a **get-current-date-time** function from definition to final sentence. Claude does not know wall-clock time. Your app does. The Messages API is the contract that lets the model *ask* for that clock without pretending it already has it.

Two artefacts you always write

1. The function

A normal function in the host language. In the demo it formatted “now” according to a format string. If nobody calls it with valid arguments, it does nothing useful. If someone does, it returns data from the *application* environment, not from model weights.

2. The tool schema

The Anthropic client expects a schema the model can read. The trainer stressed two fields first:

- **name** — which method to invoke. Keep it stable and unique. Overloaded names confuse the model.
- **description** — *when* to invoke it. This is the education layer. “Returns the current date and time formatted according to the specified format string” is enough for a clock; a vague “gets data” is not.
- **input schema** — JSON Schema for parameters (format string, city, employee id). If the function takes no arguments, the schema still exists; it is simply empty or minimal.

You then pass those schemas in the `tools` array on the Messages call, together with the user content. Binding tools is per request: you may attach one tool this turn and five the next.

TRUST CURRENT DOCS

Property names (`input_schema`, block types, beta flags) should be checked on **tool use**. Classroom screenshots of August 2026 are not a spec.

The handshake in one turn

1. User: “What is the exact time formatted as HH:MM:SS?”
2. Request includes that text plus the clock tool schema.
3. Claude **stops** with stop reason `tool_use`. It has not finished the user-facing answer.
4. The assistant message contains a **tool_use** block: tool id, name `get_current_date_time`, and model-chosen input (the format string).
5. Your code maps name → function, runs it, and obtains a result such as `7:29:31`.

6. You append a **user** message whose content is a **tool_result** block (same tool id, the result string, optional `is_error`).

7. A follow-up Messages call lets Claude write: “The exact time is 7:29:31.”

Conversation roles stay the same as earlier modules: system, user, assistant. Tool results travel as *user* content with a specialised block type so the model can bind them to the id it issued.

How Claude chooses among several tools

Lalit asked: if three tools sit in the array, who picks? **The model**, using names and descriptions (and the user text). You do not write a router in the first request. You write better descriptions. If Tool A returns errors that should trigger Tool B, put that policy in the **system prompt** and/or in B’s description so the next turn can see the error and continue.

Arvind asked whether five tools must be attached every time. No. If you know the intent, attach the subset. Attaching all of them is legal; overlapping descriptions are the failure mode.

One tool many times, or many tools in sequence

The notebook showed a `while` loop: keep calling the API until stop reason is not `tool_use`. Inside the loop, collect every `tool_use` block, run functions, pack `tool_results`, and continue.

- **Same tool twice in parallel:** “Give me the time in HH:MM and also in seconds format.” Two `tool_use` blocks, two format strings, one round of execution, then one synthesis.
- **Dependent tools:** “What time will it be 90 minutes from now?” First `get_current_date_time`, then `add_duration_to_date_time` with the returned clock plus 90 minutes. The second call needs the first result — sequential, not parallel.
- **Hard-coded dispatch:** `classroom if/else` on tool name. Production frameworks often use reflection or a registry. Either way, *your* process runs the function.

You can short-circuit in application code (inspect `is_error`, skip the model, call a fallback). Vijay’s exam-relevant default is: let Claude plan unless you have a known, cheap, deterministic branch. More LLM hops mean more hallucination surface when tool lists get large.

Schema design details that came up in Q&A

- If you want one call with many formats, describe an **array** of format strings instead of relying on two parallel calls.
- Names can be `Class.method` if your dispatcher understands that. In Python demos, a flat function name was simpler; TypeScript class style is fine if the mapping is explicit.
- **tool_choice** (including streaming/beta variants) can force or bias which tool is used. Vijay flagged the beta streaming path as useful for large tool payloads and said it was *not* an exam topic in August 2026. Confirm the live API page.

What the Agent SDK changes

The manual loop is the concept the Foundation exam cares about. In delivery you will often call the Claude Agent SDK so you do not write `while True` yourself. The model still emits `tool_use`; something still executes; results still go back. The SDK is orchestration, not a different physics.

EXAM POSTURE

If a scenario describes stop reason `tool_use`, the correct next step is: run the tool locally (or via MCP), return `tool_result`, call the API again — not invent the clock in the assistant text.

Official documentation

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Schemas, stop reasons, `tool_result`.

- **Messages API**

<https://docs.claude.com/en/api/messages>

`tools` parameter on each request.

- **Streaming**

<https://docs.claude.com/en/docs/build-with-claude/streaming>

If you combine tools with streamed tokens; check current `tool_choice` behaviour.

- **Structured outputs**

<https://docs.claude.com/en/docs/build-with-claude/structured-outputs>

Related: JSON the model must emit versus JSON a tool returns.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.